SMU Singapore Management University | School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

© MITB, SMU

Regular Expression

© MITB, SMU

Learning Objectives

- Iterators (optional)
- Generators (optional)
- Regular Expressions

© MITB, SMU

Start from a for-loop

- A simple for loop

```
list1 = [ 1, 2, 3, 4, 5 ]
for num in list1:
    print(num)
```

- Let's replace the for loop with a while loop

```
list1 = [ 1, 2, 3, 4, 5 ]
ind = 0
while ind < len(list1):
    print(list1[ind])
    ind += 1
```

Optional

© MITB, SMU

What happens if…

while loop:

- change the list to a tuple?
- change the list to a string?
- change the list to a dictionary?

It doesn't work.
Q: How would you change the code to make it print the key-value pairs?

```
list1 = [ 1, 2, 3, 4, 5 ]
ind = 0
while ind < len(list1):
    print(list1[ind])
    ind += 1
```

for loop:

- change the list to a tuple?
- change the list to a string?
- change the list to a dictionary?

It works to some degree.
Q: What if this is not what we want?
Q: If we want to print the values or key-value pairs instead?

```
list1 = [ 1, 2, 3, 4, 5 ]
for num in list1:
    print(num)
```

Optional

© MITB, SMU

More examples involving for-loop

- `zip`: zipping two lists together

```
l1, l2 = [ 1, 2, 3, 4, 5 ],
['h', 'e', 'l', 'l', 'o']
for item in zip(l1, l2):
    print(item)
```

- `enumerate`: adding orders to the items

```
l1 = ['h', 'e', 'l', 'l', 'o']
for item in enumerate(l1):
    print(item)
```

- `range`: range of evenly spaced numbers

```
for item in range(1, 10, 2):
    print(item)
```

Optional

© MITB, SMU

Different types that work with for-loop

- Check their types
- All are of different types/classes
  - `zip`
  - `enumerate`
  - `range`

```
print(type(zip(l1, l2))

print(type(enumerate(l1))

print(type(range(1, 10, 2))
```

One Important Question:
Why and how do they all work with such simple for-loop syntax?

Optional

© MITB, SMU

The answer to this question lies in …

Iterables & Iterators

Optional

© MITB, SMU

Further investigation of the types

- We never write a for loop-like this:
  - Why?

```
for item in 4:
    print(item)
```

- The common traits of those types to be used in a for-loop
  - containers that can host multiple values
  - capability to return the values one by one
- The data types with the above traits are called iterables
- Below types are all iterables:
  - list/tuple/str/dict
  - zip/enumerate/range/reversed

Optional

© MITB, SMU

Investigation into iterables

- Then it seems easy:
  - If we define a new data type and make it an iterable
  - We then can use it in the for loop
- Wait a minute, how do we make a new data type an iterable?
- In other words, how do we make it
  - contain multiple values (this seems easy to solve)
  - return its values one by one (this is the real challenge)

Optional

© MITB, SMU

The iter() function

- In Python, an **iterable** can be passed to the built-in function `iter()`, which returns some object called iterator
- First try out the `iter()` function
- We see 4 different types of iterators!

```
iter( [1, 2, 3, 4] )
iter( 'hello world' )
iter(('hello', 'world'))
iter({'former': 'hello',
'latter': 'world'})
```

- Check the help of `iter()`

```
help(iter)
```

- Try it with an integer

```
iter(8)
```

- We must make the new data type in a way that `iter()` accepts it as an iterable

Optional

© MITB, SMU

Iterator and the built-in next() function

- We still don't know how to build an iterable
- But we know we can pass an iterable to `iter()` to get an iterator
- Let's look at this iterator object and how it is used
- In fact, the iterator is the object that spits the values one by one
- To do that, let's introduce the built-in function `next()`, which yields the value from the iterator one at a time
- When all values are exhausted, an `StopIteration` error is raised.
- Check help of `next()` function

```
it = iter([4, 3, 2, 1])
print(next(it))
print(next(it))
print(next(it))
print(next(it))
```

```
print(next(it))

help(next)
```

Optional

© MITB, SMU

Summary of what we know so far

- a for-loop can get items from an iterable one by one
- `iter()` can return an iterator from an iterable
- `next()` can get items from an iterator one by one

Optional

© MITB, SMU

The answer

- Now we can answer this question:
  - Why and how do all those data types work with such simple for-loop syntax?
- The answer is that the for-loop does it by:
  - Calling `iter()` on the iterable to get the iterator, and
  - Calling `next()` on the iterator to get the values one by one
- These statements in the official document provide us some evidence
  - https://docs.python.org/3/reference/compound_stmts.html#the-for-statement

Another Important Question:
How do we make a new data type an iterable so it works with a for-loop?

Optional

© MITB, SMU

They follow …

The Iterator Protocol

Optional

© MITB, SMU

The Iterator Protocol

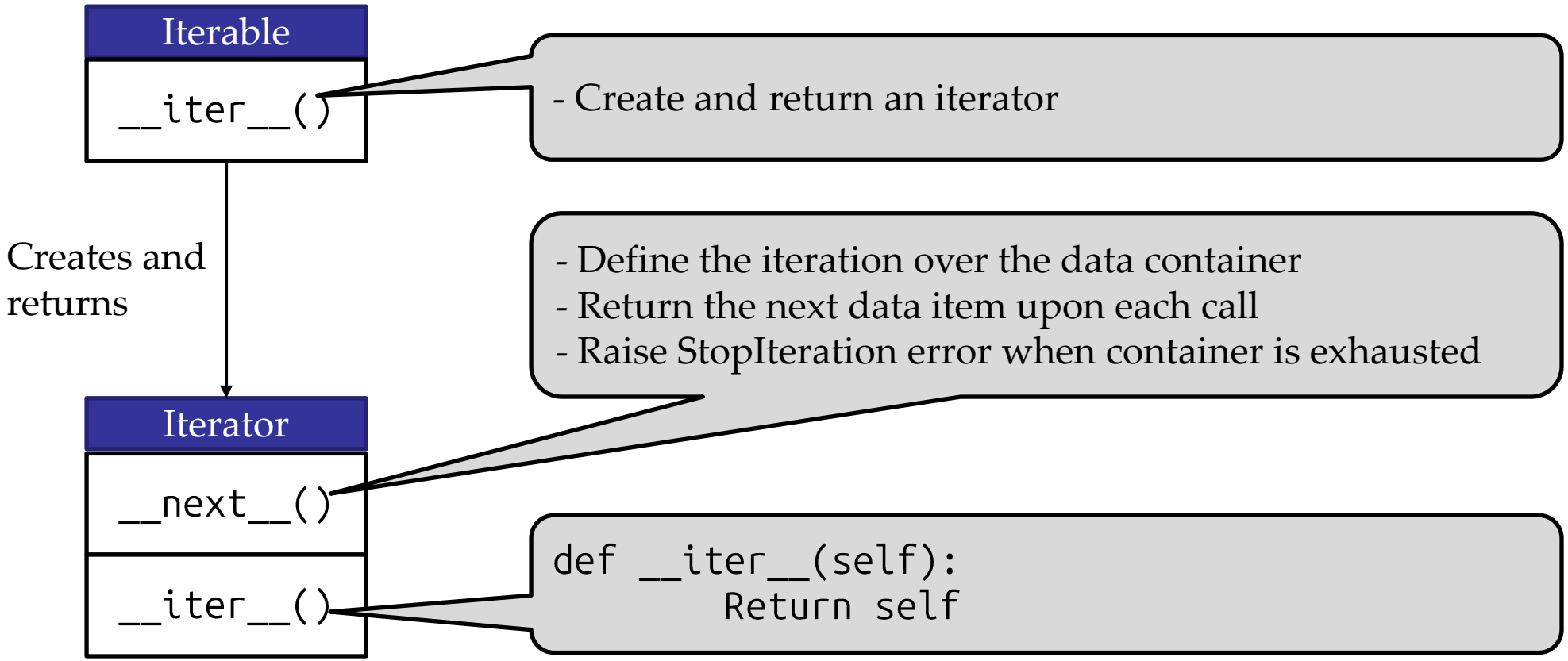
- To build an iterable, we will need to follow a rule in Python
- That rule is called **Iterator Protocol**
- The Iterator Protocol stipulates that
  - An Iterable object must have an `__iter__()` magic method to return an iterator object
  - The iterator object must have the following two magic methods
    - `__iter__()` to return the iterator object itself
    - `__next__()` to return the next item from the iterable object
- In other words, if an object is an iterable, it should have an `__iter__()` method and should be able to return a corresponding iterator in its `__iter__()` method.

Optional

© MITB, SMU

The Iterator Protocol

- Below diagram illustrates the Iterator Protocol



Optional

© MITB, SMU

Building an iterable and its iterator

- In fact, when the built-in `iter()` function is called, it calls the passed-in object's `__iter__()` function to get the new iterator object.
- When the `next()` function is called, it calls the iterator's `__next__()` function to get the next item in the container.
- In the example below, "`it`" is the iterator object return by the `__iter__()` method in the list (`[4, 3, 2, 1]`)
- And next just calls the `__next__()` method in "`it`" and gets the numbers one by one.

```
it = iter([4, 3, 2, 1])
print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
```

Optional

© MITB, SMU

The Iterator Protocol

- Now that we understand how it works, we can build an iterable and its iterator class
- First, create a class called “strStream” that contains a string we give it.
- We define the iterator “strStreamIterator” for “strStream” in a way that it return 3 characters in the string each time and the remaining characters if there are less than 3 characters left to return.

Optional

© MITB, SMU

strStream and strStreamIterator class


```
class strStream:
    def __init__(self, s):
        self.string = s

    def __iter__(self):
        return
strStreamIterator(self.string)
```

```
class strStreamIterator:
    def __init__(self, s):
        self.string = s
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= len(self.string):
            raise StopIteration
        elif self.current + 3 < len(self.string):
            chars = self.string[self.current: self.current + 3]
        else:
            chars = self.string[self.current:]
        self.current += 3
        return chars
```

Optional

© MITB, SMU

Testing the classes

- To test the `strStream` class, we create a `strStream` object "stream", call `iter()` on it to get the iterator object `strStreamIterator` and call `next()` to get the character.
- Then we use the "`stream`" object in a for loop.

```
stream = strStream("abcefgh.!3")
stream_it = iter(stream)
print(next(stream_it))
print(next(stream_it))
print(next(stream_it))
print(next(stream_it))
print(next(stream_it))

for chars in stream:
    print(chars)
```

Optional

© MITB, SMU

Summary

- The iterator protocol provides a unified interface to iterate over a container
- This interface is the `iter()` and the `next()` built-in function
- As long as the class definition follows the protocol, the objects of the class can be used in the following mechanism:
  - for loops
  - list, dict and set comprehensions
  - tuple unpacking
  - *args in function parameter list

Optional

© MITB, SMU

Now, a more practical and more concise way is …

The Generator Function

Optional

© MITB, SMU

Introducing the Generator function

- The iterator protocol is a brilliant design
- However, it is troublesome to define an iterator class for each iterable class
- The generator function provides a shortcut for implementing the same functionality
- We can see the generator mechanism as a simpler alternative of creating iterators via the commonly used function mechanism.
- A generator function helps to create a special kind of iterator called a generator to achieve this target.

Optional

© MITB, SMU

What Is a Generator function

- A generator function differences itself from a regular function by having a **yield** keyword in its definition.
- When a generator function is called, it creates a **generator** object.
- Look at below example of a generator function.
- We define a generator function `gen_func` simply to return 1, 3, 5 in each call of the function.
- In the print expression we can see that `gen_func` is a function object.

```
# generator function definition
def gen_func():
    a, b, c = 1, 3, 5
    yield a
    yield b
    yield c

# checking function definition
print(gen_func)
```

Optional

© MITB, SMU

An example Generator function

- In the print expression we can see `gen_func` returns a generator object.
- This object can be used in a for loop, just like any iterators.
- `g` is assigned to hold the generator object.
- We call next function on *g* for 4 times.
- The last call of `next()` raises a `StopIteration` error.

```
# calling a generator function
print(gen_func())

# calling generator function in for loop
for i in gen_func():
    print(i)
```

```
g = gen_func()

# apply next() function on g
print(next(g))
print(next(g))
print(next(g))
print(next(g))
```

Optional

© MITB, SMU

More on Generator function

- The last call of `next()` function on the generator object shows that the generator object behaves in the same way as iteraters.
- However, building a generator object is much easier than building an iterator class for a container.
- Next, we replace the iterator class in the previous strStream class.

Optional

© MITB, SMU

A Refined strStream w/ Generator Function

- On the right side is the new definition for the `strStream` class.
- We use a generator function to replace the `__iter__()` function.
- We also skip the definition of the Iterator class completely because the iterator will be provided by the generator function.

```
class strStream:
    def __init__(self, s):
        self.string = s

    def __iter__(self):
        i = 0
        while i < len(self.string):
            if i+3 < len(self.string):
                yield self.string[i:i+3]
            else:
                yield self.string[i:]
            i += 3
```

- We use the same test case to test the new class definition.

```
stream = strStream("abcefgh.!3")
str_gen = iter(stream)
for chars in str_gen:
    print(chars)
```

Optional

© MITB, SMU

The Yield Keyword

- We've seen how a call to a generator function will get a generator object but not a value like that to a normal function in a previous example.
- We will need to invoke the `next()` function on the generator object either explicitly – `next(generator)` – or implicitly – like in a `for` loop – to make the generator split out the values, one at a time.
- When the `next()` function is first called on the generator that the generator function is really executed – it runs from the first line of code in the function to the first yield expression, splits out the value and pauses.
- The next call of `next(generator)` will start from when it paused and pauses again when it hits the next yield expression.

Optional

© MITB, SMU

An Example of Yield Expression

- This example shows how a generator splits values when the `next()` function is called on it over and over.
- The `abc_gen()` function definition has a yield expression and returns a generator upon a call.
- `abc` is assigned with the generator returned by the `abc_gen()`.
- `abc` pauses after it executes a yield expression upon each call to `next()`.

```
def abc_gen():
    print("First letter...")
    yield 'a'
    print("Second letter...")
    yield 'b'
    print("Third letter...")
    yield 'c'

# call the abc_gen() to get the
generator
abc = abc_gen()
print(abc)

# the 3 next calls get 3 letters
print(next(abc))
print(next(abc))
print(next(abc))
```

Optional

© MITB, SMU

Generator vs. Container

- For most of the generators, we can use a normal function to return a container of all values, and this way is even more intuitive.
- However, as a benefit, the generator approach can generate infinite number of values.
- An infinite loop is used inside the `endless()` function.
- The reason is that the generator only instantiates a value when the `next()` function is called, so no infinite loop is created.
- This is called a "**lazy**" design.

```
def endless():
    i = 0
    while True:
        yield i
        i += 1

gen = endless()
print([next(gen) for i in range(10)])
print([next(gen) for i in range(10)])
print([next(gen) for i in range(10)])
```

Optional

© MITB, SMU

Generator vs. Container (Cont.)

- The generator uses less memory.
- The generator object takes only one third of the size compared to that of the list.
- The size of list will grow fast when the number of elements increases. However, the generator stays pretty much the same even when the number of elements increases.
- For large number of elements, this difference will become significant.

```
def all_letters():
    letters = []
    for i in range(65, 65 + 26):
        letters.append(chr(i))
    return letters

def gen_all_letters():
    for i in range(65, 65 + 26):
        yield chr(i)

letters_list = all_letters()
letters_gen = gen_all_letters()

import sys
print("Size of list of all capital letters:")
print(sys.getsizeof(letters_list))
print("Size of generator of all cap letters:")
print(sys.getsizeof(letters_gen))
```

Optional

© MITB, SMU

Summary

- The generator function produces a generator which is also an iterator, so we can skip the step to create an iterator class for the iterable.
- It also helps to provide a mean to produce iterator using the function mechanism. In many cases, we don't even have to build the iterator class anymore.
- By using the yield expression, we ask Python to do lot of the dirty work such as keeping track of the current status of the values, so we can focus on the code logic.
- But there is a more concise way to create a generator. We will look at it in the next section.

Optional

© MITB, SMU

Now, an even better way …

The Generator Expression

Optional

© MITB, SMU

Generator Expression

- The generator expression is a more concise way of creating a generator.
- It does not involve the use of yield expression.
- The syntax of a generator expression is very similar to that of a list/dict/set comprehension. Note that the if filtering is still available in generator expression.

  syntax: `(expression for i in iterable if condition)`

- With generator expression, we can create generators without defining and calling a generator function.
- Let's look at some examples for both generator expressions and list comprehensions to show comparison between the two.

Optional

© MITB, SMU

Generator Expression: Examples

- `gen_abc` is a generator. A call to `next()` function on it will make it generate the characters.
- `list_abc` is a list with all characters.
- If the generator expression stands by itself, the parentheses around it are necessary.
- If a function has only a generator as its parameter, the parentheses can be stripped.

```
# generator to generate 'a', 'b' and 'c'
gen_abc = (ch for ch in 'abc')
print(gen_abc)

# list comprehension
list_abc = [ch for ch in 'abc']
print(list_abc)

# parentheses are necessary here
gen_abc = ch for ch in 'abc' #SyntaxError

# some functions can take in a generator
print( list(ch for ch in 'abc') )
print( tuple(ch for ch in 'abc') )
print( sum(n for n in range(1, 10)) )
```

Optional

© MITB, SMU

The StrStream Class with Generator Exp

- We can rewrite the `__iter__` function in the `strStream` class to use a generator expression to create a generator and return it.

```
class strStream:
    def __init__(self, s):
        self.string = s

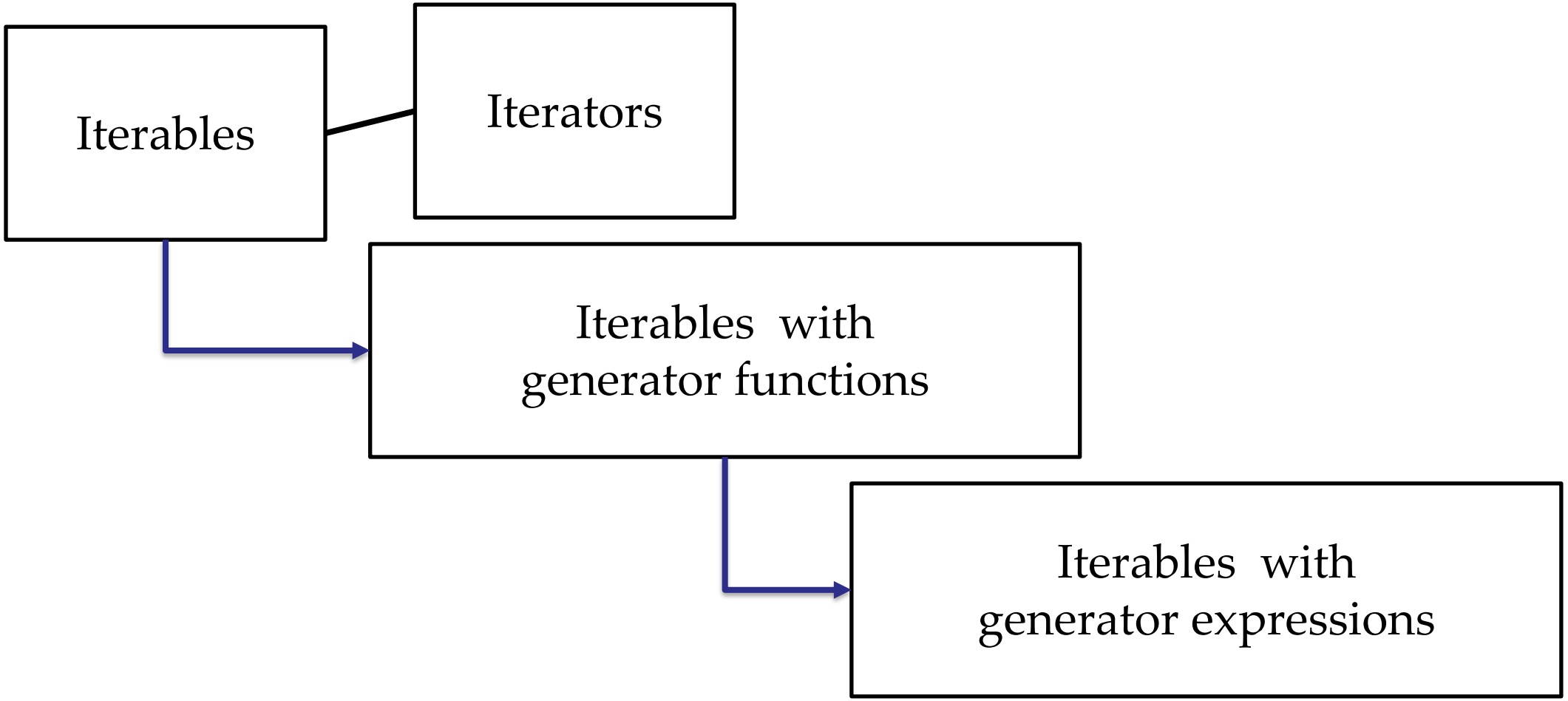
    def __iter__(self):
        return (self.string[i: i+3] for
i in range(0, len(self.string)-3, 3))
```

```
stream = strStream("abcefgh.!3")
str_gen = iter(stream)
for chars in str_gen:
    print(chars)
```

Optional

© MITB, SMU

Wrapping Up



Optional

© MITB, SMU

Regular Expression

© MITB, SMU

Text and Characters in Computers

- Most information is presented in the form of text.
- In computers, a piece of text consists of characters, with each character represented by a binary number.
- Roughly speaking, the set of characters in computers include the following
  - Letters - a to z and A to Z
  - Digits - 0 to 9
  - Punctuation marks – comma (,), colon (:), hyphen (-), etc.
  - White spaces – space, tabulation character, newline, etc.
- For some white spaces, we cannot see them when they are showed on the screen. We need some notations to represent them when we want to use them. For example, for newline, we use "\n"; and for tabulation, we use "\t".

© MITB, SMU

Finding Strings In Text With re

- One common task in data manipulation is to find certain strings or information from text, e.g., in log files, articles, or reports.
- Regular expression is a tiny programming language dedicated in such tasks.
- The search process is performed by a `re` engine.
- The `re` engine takes a **pattern string** that is set as the target for the search and applies it to search the pattern in the given text or corpus.
- Once any match is found, the re engine returns what is found in the text.



© MITB, SMU

The re Module

© MITB, SMU

Importing `re` and Building A Pattern

- The regular expression functionality is in the `re` module.
- We use `re.compile()` to compile a pattern so that the `re` engine can perform the search.
- After establishing the pattern, we can perform a search with some useful utilities.

```
import re

# compile a pattern
pat = re.compile(r'abc')

# print string representation of pat
print(pat)

# check out the class of pat
print(type(pat))
```


© MITB, SMU

Searching with match() method

match() 5
search() 6
findall() 7
finditer()

most lazy ↓ most hardworking

- The first tool is the `match()` method.
- It takes some text and tries to match the pattern from the beginning.
- Note that the `match()` method is located in the **pat** object.
- Because the match method only searches from the beginning of the text, even though there is a substring 'abc' in `mat_abc1`, the match is still None.

```
# searching the pattern with re.pattern.match
pat = re.compile(r'abc')
# 'ABC,Abc,AbC,abc' is the text in which the search is performed.
mat_abc1 = pat.match('ABC,ABc,AbC,abc')

# 'abc,ABc,AbC,abc' is the text in which the search is performed.
mat_abc2 = pat.match('abc,ABc,AbC,abc')

# check the match result.
print(mat_abc1)  # None
print(mat_abc2)  # match with position 0-3
```

(finds all occurrences)

© MITB, SMU

Searching with search() method

- match() only searches from the beginning of the text, not very useful.
- A better tool is the search() method.
- The search() method is also with pat object.
- It takes some text and tries to match the pattern in any position in the text and returns the match in re.Match class.
- But it only returns the first match.

```
# 'ABC,Abc,AbC,abc' is the text in which the search is performed.
sear_abc1 = pat.search('ABC,ABc,AbC,abc’)

# 'abc,ABc,AbC,abc' is the text in which the search is performed.
sear_abc2 = pat.search('abc,ABc,AbC,abc')

# check the match result.
print(sear_abc1)  # match with position 12-15
print(sear_abc2)  # match with position 0-3
print(type(sear_abc1))  # re.Match class
```


© MITB, SMU

Searching with `findall()` method

- The `search()` method can be used if getting only the first match is enough.
- The `findall()` method instead finds all the matched strings and return them in a list.
- Same as the previous two methods, `findall()` method is also with the `pat` object.

```
# 'ABC,Abc,AbC,abc' is the text in which
the search is performed.
find_abc1 = pat.findall('ABC,ABc,AbC,abc’)

# 'abc,ABc,AbC,abc' is the text in which
the search is performed.
find_abc2 = pat.findall('abc,ABc,AbC,abc')

# check the match result.
print(find_abc1)  # list containing 1 match
print(find_abc2)  # list of 2 matches
```


© MITB, SMU

Searching with `finditer()` method

- The `findall()` method returns all the matched strings in a list.
- If the matches in `re.Match` class are wanted or the number of matches are expected to be large, `finditer()` method can be used.
- The `finditer()` returns an iterator that lazily splits matches one at a time.
- In the last for loop, all matches in `re.Match` class are printed out.

```
# 'abc,ABc,abc,abc' is the text in which
the search is performed.
finditer_abc =
pat.finditer('abc,ABc,abc,abc')

# check the match result.
print(finditer_abc)

# prints the matches.
for m in finditer_abc:
    print(m)
```


© MITB, SMU

Summary

- We have learnt about some of commonly used re methods.
- There are still more methods or attributes that can be used.
- Up until now, the examples shown only involves matching specific characters. In the next section, we will see how the introduction of metacharacters makes re a powerful tool.

© MITB, SMU

Better put in

Metacharacters In re

cheatsheet

© MITB, SMU

Metacharacters

- Most characters only represent themselves in the re searching. For example, letters a – z and A – Z in patterns only match the same characters in the text.
- However, there are some characters that don't match themselves. They are called metacharacters.
- Below is a complete list of meta characters in re

```
. ^ $ * + ? {} [] \ | ()
```

- The metacharacters denotes the re engine to do some special searching but not literal matching. The combined use of metacharacters and normal characters gives re users great flexibility and capability to search for sequence of characters in the text.

© MITB, SMU

Different Types of Metacharacters

- The metacharacters can be categorized into several types as below:
  - Metacharacters that match a single character
  - Escaping metacharacters
  - Anchors
  - Quantifiers
  - Grouping Constructs
  - Looking Arounds (Not Required)
  - Miscellaneous Metacharacters (Not Required)

|| one class

|| next class

- We will go through first 5 types.

© MITB, SMU

Metacharacters That Match One Character

© MITB, SMU

Metacharacters That Match One Character

- This type of metacharacters only match to one character in the matching.
- When one of the metacharacters is matched to one character in the text, the re engine moves on to the next position to check the next character.
- Later you can see other types of metacharacters that don't work in this way.




```
text = "must castst"
p = re.compile(r'..st')
```

returns "must cast"

© MITB, SMU

Dot (.) Metacharacter

Dot (.)	match any single character except the newline character.

- In ex1, the text `a1\nA#` has two lines of strings. The first is `a1`, and because of the newline character after, `A#` is put on the second line.
- `a`, `1`, `A`, `#` are matched but the newline (`\n`) is not.
- In ex2, dot is combined with other characters to form a pattern.

```
# ex1
p = re.compile(r'.')
m = p.findall('a1\nA#')
print(m)   # newline (\n) is not
matched

# ex2
p = re.compile(r'.at')
m = p.findall('cat bat sat cap')
print(m)
```


© MITB, SMU

Character Class ([])

Character Class ([])

specify a set of characters to match

just need to match any one char specified

- In ex1, the pattern `[aA]` matches either a single `a` or `A`.
- No matter how many characters are in the squared brackets, they only match one character in the text, as what is returned in ex2 – all matches are single character match.
- Metacharacters lose their special meaning inside character class.

```
# ex1
p = re.compile(r'[aA]')
m = p.findall('a1\nA#')
print(m)

# ex2
p = re.compile(r'[abcABC]')
m = p.findall('abcABC')
print(m) # ['a', 'b', 'c', 'A', 'B', 'C']
```


© MITB, SMU

Using Hyphen (-) in Character Class ([])

Hyphen (-)

specify a range of characters to match
(have to be used inside character class)

- For a range of letters and digits, a hyphen (-) can be used.
- In ex3, `a-z` covers all lowercase letters and `0-9` covers 0 to 9.
- You can choose your range with hyphen as shown in ex4.
- If you want to match a literal hyphen, put it in the beginning or the end inside [].

```
# ex3
p = re.compile(r'[a-z0-9]')
m = p.findall('d0A3z6P')
print(m)

# ex4
p = re.compile(r'[C-Eb-c2-4]')
m = p.findall('ABCDEF abcd 012345')
print(m)

# ex5
p = re.compile(r'[-a-e]') # or [a-e-]
m = p.findall('e-a-s-y, easy')
print(m)
```


© MITB, SMU

Caret (^) in Character Class ([])

caret (^)	match any character not in the character set (have to be used inside character class)

- By putting a caret(^) in the beginning inside the character class, we can match any character NOT in the character class.
- In ex6, the pattern excludes all digits.
- caret not in the beginning works as a normal character. (ex7)
- A caret outside a character class has a different meaning.

```
# ex6: [^]
p = re.compile(r'[^0-9a-z]')
m = p.findall('1 2 3 Go')
print(m)  # [' ', ' ', ' ', 'G']

# ex7:
p = re.compile(r'[0-9^a-z]')
m = p.findall('1 2 3 ^Go') # Note the
^ character
print(m)  # ['1', '2', '3', '^', 'o']
```


© MITB, SMU

Digits: \d and \D

\d \D	match based on whether a character is a digit

- \d matches a character that is a numeric digit. (\d = [0-9])
- \D matches a character that is NOT a digit. (\D=[^0-9])
- \d and \D complement each other in the match. Such relationship also applies to \w\W, \s\S, \b\B.
- In ex2, we can see that the newline is also matched by \D.

```
# ex1
p = re.compile(r'\d')
m = p.findall('a1\nA#')
print("\d matches: ", m)

# ex2
p = re.compile(r'\D')
m = p.findall('a1\nA#')
print("\D matches: ", m)
```


© MITB, SMU

Word Characters: \w and \W

\w \W	match based on whether a character is a word character

- \w matches a character that is a letter, a digit or an underscore (_)
- \W matches a character that is NOT any of the above.
- \w = [a-zA-Z0-9_]
- \W = [^a-zA-Z0-9_]

```
# ex1
p = re.compile(r'\w')
m = p.findall('_#a!E$4-')
print("\w matches: ", m)
# ['_', 'a', 'E', '4']

# ex2
p = re.compile(r'\W')
m = p.findall('_#a!E$4-')
print("\W matches: ", m)
# ['#', '!', '$', '-']
```


© MITB, SMU

Whitespaces: \s and \S

\s \S	match based on whether a character is a whitespace

- \s matches a whitespace character
- \S matches a character that is NOT a whitespace character.
- Common whitespace characters include space ( ), tabulation (\t), newline (\n) and carriage return (\r)

```
text = 'Name\tISSS610\nJoe Jones\tA\n'

# ex1
p = re.compile(r'\s')
m = p.findall(text)
print("\s matches: ", m)

# ex2
p = re.compile(r'\S')
m = p.findall(text)
print("\S matches: ", m)
```


© MITB, SMU

Wooclap!

What is the output of the below code?

```
text = "boo"

p = re.compile(r'.o')
print(p.findall(text))
```

a. ['bo']
b. ['bo', 'oo']
c. ['boo']
d. ['bo', 'o']

Escaping Metacharacters

© MITB, SMU

Backslash (\) as Escaping Metacharacter

Backslash (\)	Removes the special meaning of a metacharacter. (have to be used before a metacharacter)

- We've seen \w\W, \d\D and \s\S to represent a particular set of characters.
- Backslash (\) itself is also a metacharacter.
- Its use is to remove (escape) the special meaning of other metacharacters.
- By putting backslash in front of a metacharacter, we can pertain the literal meaning of a metacharacter.
- re engine will match the character using its literal meaning in this case.
- For example, "\." matches a period (".") but not any one character.
- The escape character can remove the meaning of itself, i.e., "\\".

© MITB, SMU

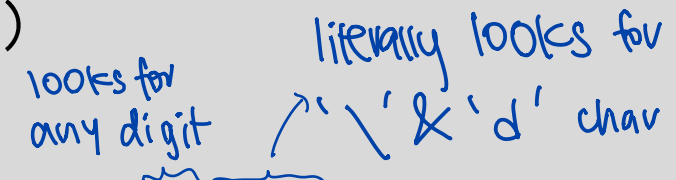
Backslash (\) as Escaping Metachar - ex

- In ex1, p1 matches any character while p2 matches a dot.
- m1 contains the first match 's' in position 0 and m2 contains the first match '.' in position 3.

- In ex2, the first '\d' matches to any digit, but the second '\d' is escaped by the extra backslash before – it matches a '\' followed by a 'd'.

```
# ex1
p1 = re.compile('.')
p2 = re.compile('\.')
m1 = p1.search('smu.edu.sg')  # 's'
m2 = p2.search('smu.edu.sg')  # '.'
print(m1, m2)

# ex2
p = re.compile(r'\d\\d')
m = p.findall('135\d')
print(m)
```



© MITB, SMU

The Backslash Plague

- We have learnt about how re uses backslash to escape the special meaning of metacharacters.
- However, Python also use backslash in a similar way to deal with some special characters in Python strings!
- Since we use re in the context of Python, this could potentially cause conflict between the re functionality and python strings.
- Next are some demonstration of this backslash plague.

© MITB, SMU

The Backslash Plague – Demo1

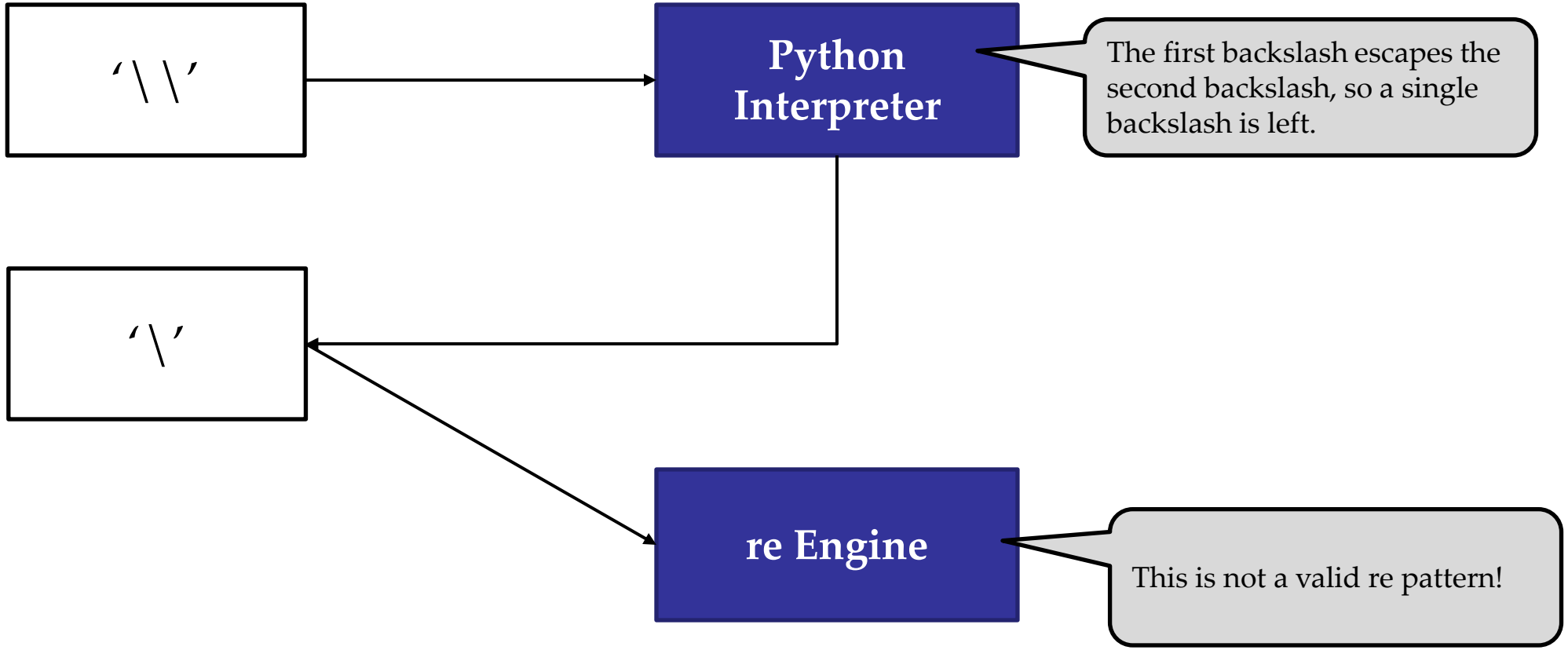
- In demo1, we attempt to match the "\" in the given path.
- However, the `re.compile` command raises a "bad escape" error.

```
# demo1
p = re.compile('\\')
m = p.findall('D:\Courses\Python')
print(m)
```

- The problem is that the Python interpreter processes the "\\" before re does. Python also use backslash to bypass special meaning of a backslash in Python string.
- Before re sees the pattern, Python reduces the '\\' to a single '\'!
- You can confirm this behavior with a print function: print('\\').
- When re sees the pattern, it becomes a single '\'. However, a single '\' is not a valid pattern in re, so it raises a bad-escape error.

© MITB, SMU

Illustration of Two-Level Intepretation



© MITB, SMU

Using Python Raw String As re Pattern

- One practical way to solve this problem is to use the python raw string to form an re pattern.
- The raw string does not change much from a normal string – a lowercase 'r' is leading a normal double/single-quoted string to indicate it is a raw string.
- For example, `r"this is a raw string."` or `r'raw string'`
- Python interpreter does not interpret raw strings. In the previous example, while '\\' is reduced by the Python interpreter to a single backslash, `r'\\'` is passed to re unchanged.
- In the printout, the matched single backslash is adjusted into a '\\' format to avoid ambiguity.

```
# demo2
p = re.compile(r'\\')
m = p.findall('D:\Courses\Python')
print(m)
```

always put 'r' to indicate raw string

© MITB, SMU

Conclusion

- In practice, it is recommended that we …

Always use raw string for re pattern!!

© MITB, SMU

Anchors

© MITB, SMU

Anchors – Zero-width Matches

- Anchors do not match actual characters in the text.
- They match zero-width positions, such as start or end of a line, or a position between characters.
- In matching characters in a pattern to actual characters in the text, after each successful match, the re engine advances to the next character to performance the next matching. This is called consuming a character in the text.
- However, anchors don't consume any characters.
- We will talk about below anchors in this section:
  - ^: beginning of text
  - $: end of text
  - \b, \B: word boundary

© MITB, SMU

Caret (^)

- We have seen a caret used in a character class. Here the caret is used without a character class.
- It matches the starting position in the text.
- In the case of Multiline text, caret only matches the starting position of the text but not each line.
- This behavior can be changed with re flag `re.MULTILINE` or `re.M`. More will be discussed on the flags later.

```
# ex1
p = re.compile(r'^ab.')
m = p.findall('abc abd abe abf')
print(m)  # ['abc']

# ex2
p = re.compile(r'^a[ab]c')
m = p.findall('''aac\nabc''')
print(m)  # ['aac']

p = re.compile(r'^a[ab]c', re.M)
m = p.findall('''aac\nabc''')
print(m)  # ['aac', 'abc']
```

→ accepts if match at newline

Dollar ($)

- The dollar sign matches the ending position in the text.
- Similar to caret, dollar sign matches the ending position but not in each line in multiline text, but this behavior can also be changed with 're.MULTILINE' or 're.M'

```
# ex1
p = re.compile(r'ab.$')
m = p.findall('abc abd abe abf')
print(m)  # ['abf']

# ex2
p = re.compile(r'[ab]c$')
m = p.findall('''ac\nbc''')
print(m)  # ['bc']

p = re.compile(r'[ab]c$', re.M)
m = p.findall('''ac\nbc''')
print(m)  # ['ac', 'bc']
```

match at end of the line

answer must match length = 2

search for 'ac' or 'bc' at the end

© MITB, SMU

Word Boundaries: \b and \B

\b \B	Match based on whether a position is a word boundary

- A word in re consists of a sequence of `\w` (`[a-zA-Z0-9_]`)
- `\b` matches the beginning or end position of a word, that is, a zero-width position between `\w` and `\W`.
- In ex1, the pattern matches two digits standing by themselves.
- In ex2, the pattern matches two word-characters.

```
# ex1
p = re.compile(r'\b\d\d\b')
m = p.findall('1 2 3 11 12 13')
print(m)
# ['11', '12', '13']

# ex2
p = re.compile(r'\b\w\w\b')
m = p.findall('aa,ab;ac(AA)AB AC')
print(m)
# ['aa', 'ab', 'ac', 'AA', 'AB', 'AC']
```

Quantifiers

© MITB, SMU

Quantifiers

- It indicates the repetition of a part of the re pattern before it.
- To include a quantifier in an re pattern, there must be a preceding partial pattern, to which a quantifier can attach.
- We will go through below quantifiers:
  - *: zero or more repetitions of the preceding partial pattern.
  - ?: zero or one repetition of the preceding partial pattern.
  - +: one or more repetitions of the preceding partial pattern.
  - {m}: m repetitions of the preceding pattern
  - {m, n}: any number of repetitions of the preceding partial pattern from m to n, inclusive.

© MITB, SMU

Asterisk (*), Question Mark (?) and Plus (+)

*	Match zero or more repetitions of the preceding pattern
?	Match zero or one repetition of the preceding pattern
+	Match one or more repetitions of the preceding pattern

- The three quantifiers are very similar.
- They can be combined with normal characters, or one-to-one match metacharacters, such as 'a?', or '\w*', or '[a-d1-5]+'
- Run code from ex1 and you can see the difference among the three quantifiers.

```
text = 'a ab ac abc aac aabc aaac ababc'

p = re.compile(r'a[ab]*c')
m = p.findall(text)
print(m)

p = re.compile(r'a[ab]+c')
m = p.findall(text)
print(m)
# abc, aac, aabc, aaac, ababc
p = re.compile(r'a[ab]?c')
m = p.findall(text)
print(m) # ac, abc, aac, abc, aac abc
```

can have 0/1/many repetitions of 'a','b','ab'

most lenient

must have ≥1 a or b (can have both)

has either 0 or 1 'a' or 'b', not both

© MITB, SMU

{m} and {m,n}

{m}	Match exactly m repetitions of previous pattern
{m,n}	Match m to n repetitions of previous pattern, inclusive

range.

- '{m}' and '{m,n}' give re the ability to match exact number or a range of times to sub-patterns.
- Note that in '{m,n}' there should not be any space between the coma and n.
- If there is no lower or upper limit, {,n} and {m,} can be used respectively.

```
text = '1 2 3 11 12 13 111 112 113 567890'

p = re.compile(r'\d{3}')
m = p.findall(text)
print(m)
#111,112,113,567,890
p = re.compile(r'\d{2,3}')
m = p.findall(text)
print(m)
```

includes 3 itself

p = re.compile(r'\d{,3}')
m = p.findall(text)
print(m) → returns everything include spaces.

© MITB, SMU

Wooclap!

What is the output of the below code?

```
text = "hello world"

p = re.compile(r'\w*')
print(p.findall(text))
```

0 or more word chars

a. ['hello world']
b. ['hello', 'world']
c. ['hello', 'world', '']
d. ['hello', '', 'world', '']

Grouping Constructs

© MITB, SMU

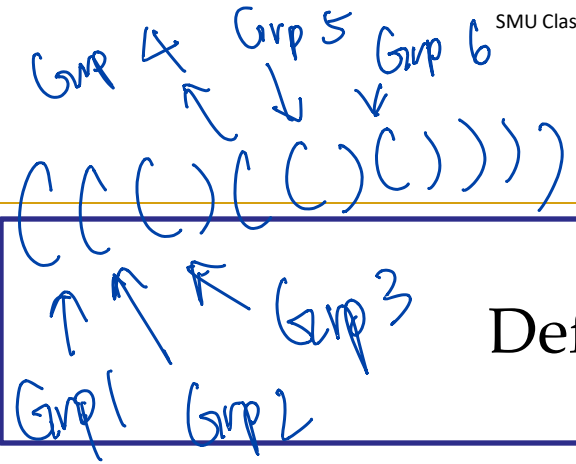
Grouping and Backreference



- The grouping construct is a powerful tool through which you can put different parts of the whole re pattern into groups.
- Grouping can also help pertain certain information out of the whole match.
- The grouping construct is done with parentheses ('()').
- Combining grouping with quantifiers, we can match repetitive groups.
- Alternation (|) can be applied within groups to act similarly to an "OR" operator.
- Another powerful feature that builds on grouping is backreference, which refers back to a previous group with the same literal string.
- In this section we will go through grouping with numbered groups, alternation and backreference to numbered groups.

© MITB, SMU

Groups (())



Groups (())

Define sub-pattern or group

- In ex1, the re engine matches the whole pattern including the colon.
- The sub-patterns in the two groups are kept and returned by `findall()`.
- We can keep only the information we want and remove useless characters in this way.

literal colon
≥1 word char
at least ≥1 digits.

```
# ex1
p = re.compile(r'(\w+): (\d+)')
m = p.findall('Course: Grade\nMath: 89\nPhysics: 92\n English: 78')
print(m)
# [('Math', '89'), ('Physics', '92'), ('English', '78')]
```

1st match
2nd match
3rd match

match = "Physics : 92"
subject, score = match.split(': ')
returns "Physics", '92'

len(m) ⇒ number of matches, not length of tuple

Backtesting is James's fav to test

© MITB, SMU

Combining Groups With Quantifiers

- When a quantifier follows a group, the quantifying effect applies to the whole group.
- In ex2, we match one or more repetition of “ha”. However, in the “hahahaha” case, we match all 4 repetitions of “ha”, but since the group is on “ha” only, only 1 is returned.
- Ex3 is a more realistic example in which chapter titles are matched.

only 1 group because only 1 ( )

```
# ex2
p = re.compile(r'(ha)+')  # ha | haha | hahaha | ....
m = p.findall('ha hh aa hahahaha')
print(m)  # ['ha', 'ha']

# ex3
chapters = 'Chapter 12: Numpy\n\
Chapter 13: Pandas\n\
Chapter 14: Data Visualzation'

p = re.compile(r'^Chapter (\d+: .+)',
re.M)
m = p.findall(chapters)
print(m)
# ['12: Numpy', '13: Pandas', '14:
Data Visualzation']
```

must start w/ “Chapter”

treat new line as new start

© MITB, SMU

Wooclap!

What is the output of the below code?

```
text = r"hahahehahahe"
p = re.compile(r'(h[ae]+)')

print(p.findall(text))
```

a. ['ha', 'he', 'ha', 'he']
b. ['ha']
c. ['hahahehahahe']
d. ['he']

Alternation (|)

Alteration (|)

Match the sub-pattern before or the one after

- The alternation (|) is usually used within a group to indicate a set of alternatives to match in the group.
- It can also be used outside of a group.
- In ex1, the pattern tries to match all files with a suffix of “exe”, “bat” or “zip”.

```
# ex1
p = re.compile(r'(\w+)\.(bat|zip|exe)')
m = p.findall('game.exe auto.bat
text.zip')
print(m)
# [('game', 'exe'), ('auto', 'bat'),
('text', 'zip')]
```


© MITB, SMU

Retrieving Matched Groups

- The `re.Match.findall()` method always returns all the groups in a list.
- However, the `re.Match` class also provide us some utilities to retrieve the matched groups.
- Those utilities include the following:
  - `re.Match.groups()`
  - `re.Match.group()`
- We can apply these utilities after we get the `re.Match` objects from below functions that we have talked about.
  - `re.Pattern.match()`
  - `re.Pattern.search()`
  - `re.Pattern.finditer()`

© MITB, SMU

re.Match.groups()

- The `groups()` function is straightforward and returns all matched groups.
- The `groups()` function is straightforward and returns all matched groups.
- In ex1, the pattern specifies two groups to match every two file names.
- `p.search()` finds out the first two file names and return the two groups in a tuple.

```
# ex1
p = re.compile(r'(\w+\.\w+)\s(\w+\.\w+)')
m = p.search('game.exe auto.bat
text.zip')
print(m)   # the re.Match object
print(m.groups())
# ('game.exe', 'auto.bat')
```


© MITB, SMU

re.Match.group()

- The `group()` function allows users to choose different groups by giving the indices of the groups.
- When we define a group in the pattern, this group is assigned an index.
- Then index 1 refers to the first group, 2 refers to the second, and so on.
- However, index 0 is predefined as the entire match to the pattern, and we cannot change that.
- `group(0)` returns the whole match.
- `group(2)` returns the second captured group.
- `group(2, 3, 4)` returns the 2nd, 3rd and 4th groups.
- Let's see some examples.

© MITB, SMU

re.Match.group() example

- In ex1, the pattern specifies 4 groups.
- `group(0)` returns the entire match.
- `group(1)` returns the first word
- `group(2, 3, 4)` returns a tuple containing 2nd, 3rd and 4th words.

```
# ex1

pattern = r'(\w+)\W+(\w+)\W+(\w+)\W+(\w)+'
p = re.compile(pattern)
m = p.search('one,,,two:three++++++4')

print(m.group(0))
# one,,,two:three++++++4

print(m.group(1))
# one

print(m.group(2, 3, 4))
# ('two', 'three', '4')
```


© MITB, SMU

Backreference (\1, \2, \3, ...)

Backreference (\1, \2, \3, ...)

Match the content of a previously captured group

- Now that you have learnt about the group numbers, it is easy to introduce backreference.
- If a group repeats itself later in the same pattern, with backreference, we are able to match this group.
- Note that `'(\w+)-\1'` is different from `'(\w+)-\w+'` in that when the first group is matched, `\1` match the same literal string in group1. For example, the two patterns both match 'one-one', but the one with backreference won't match 'one-two'.

© MITB, SMU

Backreference Example

- In ex1, the pattern tries to match the type of number that starts with a few digits followed by one digit and then repeats the first few digits.
- The `finditer` function will search for the pattern throughout the given text and return an iterator that spits all the matches one by one.
- We then can put the iterator in a for loop and call the `group()` method on each `re.Match` object.

```
# ex1

p = re.compile(r'((\d+)\d\2)')
m = p.finditer('1234123, 11311,
123, 54345')
for string in m:
    print(string.group(1, 2))

# ('1234123', '123')
# ('11311', '11')
# ('434', '4')
```


© MITB, SMU

Flags

© MITB, SMU

Flags For re

- We have seen how the flag `re.MULTILINE` or `re.M` can change how "^" and "$" are matched in the searching.
- Flags can control different ways the re engine works when performing search.
- Three common flags that are very useful are:
  - `re.MULTILINE` or `re.M`: make "^"/"$" match starting/ending position of each line.
  - `re.IGNORECASE` or `re.I`: match letters in a case-insensitive way.
  - `re.DOTALL` or `re.S`: make "." match any character, including newlines.

© MITB, SMU

Flags Example

- In ex1, `re.I` makes re engine match all strings without considering upper or lower cases.
- In ex2, `re.S` makes "." a truly universal metacharacter that matches any character.

```
# ex1
p1 = re.compile(r'abc')
m1 = p1.findall('abc ABC aBC Abc')
p2 = re.compile(r'abc', re.I)
m2 = p2.findall('abc ABC aBC Abc')
print(m1)  # ['abc']
print(m2)  # ['abc', 'ABC', 'aBC', 'Abc']

# ex2
p1 = re.compile(r'.')
m1 = p1.findall('Aa1! \n')
p2 = re.compile(r'.', re.S)
m2 = p2.findall('Aa1! \n')
print(m1)  # ['A', 'a', '1', '!', ' ']
print(m2)  # ['A', 'a', '1', '!', ' ', '\n']
```


© MITB, SMU

Exercise 7-1: Regular Expression Warm Up

- In this exercise, remove the underline and fill in your pattern and/or re flag. No other code should be added.

© MITB, SMU

Exercise 7-1a

- Find out all positive integers from 10 to 999 inclusive from the given piece of text.

```
5 10 999 1314 666 0 1 +3 1000 000 0123 1024 11 +99 100 001 010
09 9999 000134 +402 123 +1000 +000999
```

- Leading zeros in an integer should be disregarded when matching, but should be included in the match
- Plus sign denotes a positive integer and should be included in the match

© MITB, SMU

Exercise 7-1b

- Find out all the words that are longer than 7 characters in the given text.

```
Regular expression is one of the useful tools a programmer
should try to master. Although regular expression can be
confusing at first, it is very compact and elegant.
```


© MITB, SMU

Exercise 7-1c

- Find out all the words containing at least one hyphen ("-") in the given text.

```
The man is good-looking and rich.
The eleven-year-old twenty-five-storey building was developed by
a famous developer in town.
This art piece is one-of-a-kind.
There is a five-and-one-half-foot-long sign at the outskirt of
the town.
```


© MITB, SMU

Exercise 7-1d

- Extract all the sentences spoken by the Female from the given dialogue.

```
Female: How are you?
Male: It's been three years. I couldn't stop thinking about
you.
Female: I am sorry, but I am already married to someone else.
Male: I don't care!
Female: Please don't be like this. I am married.
Male: I said I don't care! I just want my money back!
```


© MITB, SMU

Exercise 7-1e

- Find out all the words containing the same letter at least three times from the given text.

```
In the event that the Purchaser defaults in the payment of any instalment of purchase price, taxes, insurance, interest, or the annual charge described elsewhere herein, or shall default in the performance of any other obligations set forth in this Contract, the Seller may: at his option: (a) Declare immediately due and payable the entire unpaid balance of purchase price, with accrued interest, taxes, and annual charge, and demand full payment thereof, and enforce conveyance of the land by termination of the contract or according to the terms hereof, in which case the Purchaser shall also be liable to the Seller for reasonable attorney's fees for services rendered by any attorney on behalf of the Seller, or (b) sell said land and premises or any part thereof at public auction, in such manner, at such time and place, upon such terms and conditions, and upon such public notice as the Seller may deem best for the interest of all concerned, consisting of advertisement in a newspaper of general circulation in the county or city in which the security property is located at least once a week for Three (3) successive weeks or for such period as applicable law may require and, in case of default of any purchaser, to re-sell with such postponement of sale or resale and upon such public notice thereof as the Seller may determine, and upon compliance by the Purchaser with the terms of sale, and upon judicial approval as may be required by law, convey said land and premises in fee simple to and at the cost of the Purchaser, who shall not be liable to see to the application of the purchase money; and from the proceeds of the sale: First to pay all proper costs and charges, including but not limited to court costs, advertising expenses, auctioneer's allowance, the expenses, if any required to correct any irregularity in the title, premium for Seller's bond, auditor's fee, attorney's fee, and all other expenses of sale occurred in and about the protection and execution of this contract, and all moneys advanced for taxes, assessments, insurance, and with interest thereon as provided herein, and all taxes due upon said land and premises at time of sale, and to retain as compensation a commission of five percent (5%) on the amount of said sale or sales; SECOND, to pay the whole amount then remaining unpaid of the principal of said contract, and interest thereon to date of payment, whether the same shall be due or not, it being understood and agreed that upon such sale before maturity of the contract the balance thereof shall be immediately due and payable; THIRD, to pay liens of record against the security property according to their priority of lien and to the extent that funds remaining in the hands of the Seller are available; and LAST, to pay the remainder of said proceeds, if any, to the vendor, his heirs, personals representatives, successors or assigns upon the delivery and surrender to the vendee of possession of the land and premises, less costs and excess of obtaining possession.
```


© MITB, SMU

Exercise 7-2: ASCII Art

- Our task is to decompress a list of compressed strings.
- Each compressed string is a combination of a character followed by a number. This number indicates how many times the character repeats after decompression.
- E.g., the string `'!1%3*2'` means that we print:
  - 1 `'!'`
  - 3 `'%'`
  - 2 `'*'`
- This eventually prints `'!%%%**'`

© MITB, SMU

Exercise 7-2: ASCII Art

- In this question, given a list of compressed strings, we want to decompress the strings and print each of them out in separate lines.
- For example, given:

```
[' 5_1', ' 4/1 1\1', ' 3/1 1_1 1\1', ' 1/1 1_3 1\1', '/1_1/1
                                    3\1_1\1']
```

- We get the below pattern:

```
    _
   / \
  / _ \
 / ___ \
/_/   \_\
```


© MITB, SMU

(recap) Different Types of Metacharacters

- The metacharacters can be categorized into several types as below:
  - Metacharacters that match a single character
  - Escaping metacharacters
  - Anchors
  - Quantifiers
  - Grouping Constructs
  - Looking Arounds (Not Required)
  - Miscellaneous Metacharacters (Not Required)
- We will go through first 5 types.

© MITB, SMU

Exercise 7-3: Supervised Spam Filter

- In this exercise, we are given a training corpus containing email messages which are labelled as spam / not spam.
- This is an example of a supervised problem – we are provided with ground truth labels for each email message.
- For example, the below shows some examples of our data:

Message	Label
"Congratulations! You've won a free ticket to Bahamas!"	spam
"Reminder: Your invoice is due tomorrow."	not spam
"Hurry up and claim your $1000 gift card now!"	spam
"Don't miss out on this amazing opportunity to make money fast!"	spam

© MITB, SMU

Exercise 7-3: Supervised Spam Filter

- In addition to the corpus, we are also given another set of similar data, which will be used to test the performance of our spam filter.
- Our goal is to design a spam filter using regular expressions from the training corpus to detect whether the given message is a spam message or not.
- We will use the below strategy to filter the spam messages:
  - Step 1: Collect all individual words inside the training corpus and count the frequency of each word in “spam” vs “not spam”.
  - Step 2: Identify the top N words that appear within the “spam” words which are not present in the top 2 * N words within the “not spam” words.
  - Step 3: Observe the results from Step 2. Figure out some words that we should first remove from our individual words and remove them.
  - Step 4: Compile a regular expression that combines these patterns into a single regex.
  - Step 5: Given a new message, search for this pattern. If found, the message is “spam”.
  - Step 6: We can calculate our accuracy of our model, by taking the ratio of correct predictions to the total number of predictions.

© MITB, SMU



Questions?

Thank You

© MITB, SMU